Introducción a la Programación

Algoritmos y Estructuras de Datos I

Segundo cuatrimestre de 2026

Departamento de Computación - FCEyN - UBA

Práctica 5: Recursión sobre listas

Repaso de Listas en Haskell

1. ¿Cómo se construye la lista vacía?
2. ¿Cómo se construye la lista con al menos 1 elemento?
3. La representación visual de una lista con 3 elementos es: [5,7,2]. ¿Cómo se construye esta lista usando los constructores de Haskell? (el detrás de escena)
4. ¿Cómo represento en general, con parámetros, una lista con al menos 2 elementos?

Longitud

`longitud :: [t] -> Integer`, que dada una lista devuelve su cantidad de elementos.

`problema longitud` (s: $seq\langle T \rangle$) : $\mathbb{Z}$ $\{$
  `requiere:` $\{ True \}$
  `asegura:` $\{ resultado =$ cantidad de elementos de s $\}$
$\}$

Sumatoria

```
problema sumatoria (s: seq⟨ℤ⟩) : ℤ  {
  requiere: { True }
  asegura: { resultado = ∑_{i=0}^{|s|-1} s[i] }
}
```

En grupos

1. Cada grupo resuelve un problema (en papel).
2. El grupo debe ponerle un nombre representativo a la función teniendo en cuenta la especificación.
3. Una vez resuelto le damos el papel al equipo de “al lado”.
4. El equipo debe revisar el código y sugerir posibles mejoras sobre la legibilidad y modularización del mismo.
5. Decidir si el nombre que le puso el grupo a la función es apropiada para la especificación.
6. Puesta en común en el pizarrón.

Ejercicio 2.5

Implementar `quitar :: (Eq t) => t -> [t] -> [t]`, que dados un elemento e y una lista s, elimina la primera aparición de e en la lista s (de haberla).

Posible solución Ej2.5

```
quitar :: (Eq t) => t -> [t] -> [t]
quitar _ [] = []
quitar e (x:xs) | e == x = xs
                | otherwise = x: quitar e xs
```

Ejercicio 3.3

Definir las siguientes funciones sobre listas de enteros

`problema máximo` (s: $seq\langle\mathbb{Z}\rangle$) : $\mathbb{Z}$ $\{$
  `requiere:` $\{ |s| > 0 \}$
  `asegura:` $\{ resultado \in s \wedge$ todo elemento de s es menor o igual a $resultado\}$
$\}$

Posible solución Ej3.3

```
maximo :: [Int] -> Int
maximo [x] = x
maximo (x:xs) | x > maximo xs = x
              | otherwise = maximo xs
```

Ejercicio 3.9

Definir las siguientes funciones sobre listas de enteros

`problema ordenar` (s: $seq\langle\mathbb{Z}\rangle$) : $seq\langle\mathbb{Z}\rangle$ {
  `requiere:` { True }
  `asegura:` { $resultado$ contiene los elementos de s ordenados de forma creciente}
}

Sugerencia: Hay muchas formas distintas de ordenar secuencias. Una opción puede ser utilizar la función `máximo` y la función `quitar` que ya implementamos.

Posible solución Ej3.9

```
ordenar :: [Int] -> [Int]
ordenar [] = []
ordenar xs = (ordenar (quitar max xs)) ++ [max]
             where max = maximo xs
```

Ejercicio 6

Usando los siguientes tipos de datos:

- `type Texto = [Char]`
- `type Nombre = Texto`
- `type Telefono = Texto`
- `type Contacto = (Nombre, Telefono)`
- `type ContactosTel = [Contacto]`

Ejercicio 6

Usando los siguientes tipos de datos:

- `type Texto = [Char]`
- `type Nombre = Texto`
- `type Telefono = Texto`
- `type Contacto = (Nombre, Telefono)`
- `type ContactosTel = [Contacto]`

a) Implementar una función que me diga si una persona aparece en mi lista de contactos del teléfono: `enLosContactos :: Nombre -> ContactosTel -> Bool`

Ejercicio 6

Usando los siguientes tipos de datos:

- `type Texto = [Char]`
- `type Nombre = Texto`
- `type Telefono = Texto`
- `type Contacto = (Nombre, Telefono)`
- `type ContactosTel = [Contacto]`

a) Implementar una función que me diga si una persona aparece en mi lista de contactos del teléfono: `enLosContactos :: Nombre -> ContactosTel -> Bool`

b) Implementar una función que agregue una nueva persona a mis contactos, si esa persona está ya en mis contactos entonces actualiza el teléfono. `agregarContacto :: Contacto -> ContactosTel -> ContactosTel`

Posible solución Ej6

```haskell
type Texto = [Char]
type Nombre = Texto
type Telefono = Texto
type Contacto = (Nombre, Telefono)
type ContactosTel = [Contacto]

enLosContactos :: Nombre -> ContactosTel -> Bool
enLosContactos _ [] = False
enLosContactos nombre ((contactoNombre, _):contactos) = nombre == contactoNombre
                                    || enLosContactos nombre contactos

agregarContacto :: Contacto -> ContactosTel -> ContactosTel
agregarContacto c [] = [c]
agregarContacto (nombre, nuevoTel) ((contactoNombre, contactoTel):contactos)
                | contactoNombre == nombre = (nombre, nuevoTel):contactos
                | otherwise = (contactoNombre, contactoTel): (agregarContacto
                (nombre, nuevoTel) contactos)
```

Ejercicio 8

En este ejercicio vamos a trabajar con matrices.
Vamos a representar una matriz como una secuencia de secuencias.

Si m es nuestra secuencia de secuencias que representa una matriz, la secuencia i-ésima de m representa la i-ésima fila de la matriz, y el elemento j-ésimo dentro de la secuencia i-ésima representa el elemento en la fila i, columna j de la matriz.
Por ejemplo, a la matriz identidad de $\mathbb{R}^3$ la podemos definir como la lista de listas: $[[1, 0, 0], [0, 1, 0], [0, 0, 1]]$ en Haskell.

Usando esta representación, definir las siguientes funciones sobre matrices:

Ejercicio 8 (Similar al ejercicio 8.1)

`multiplicarFilas :: [[Integer]] -> [Integer]` según la siguiente especificación:

`problema` `multiplicarFilas` (m: $seq\langle seq\langle \mathbb{Z} \rangle\rangle$) : $seq\langle \mathbb{Z} \rangle$ {

`requiere:` { $|m| > 0$ }

`requiere:` { $|m[0]| > 0$ }

`requiere:` { Todos los elementos de la secuencia m tienen la misma longitud }

`asegura:` {$|res| = |m|$}

`asegura:` {Para todo $0 \leq i < |res|$, $res[i] = \prod_{j=0}^{|m[0]|-1} m[i][j]$}

}

Por ejemplo, `multiplicarFilas [[1,2],[3,4]]` es igual a `[2,12]`.

Posible solución Ej8 (Similar al 8.1)

```
multiplicarFilas :: [[Integer]] -> [Integer]
multiplicarFilas [] = []
multiplicarFilas (x:xs) = (multiplicarFila x) : multiplicarFilas xs

multiplicarFila :: [Integer] -> Integer
multiplicarFila [x] = x
multiplicarFila (x:xs) = x * multiplicarFila xs
```

Ejercicio 8.2

Implementar la función `cantidadDeApariciones :: Integer -> [[Integer]] -> Integer` según la siguiente especificación:

`problema cantidadDeApariciones` (e: $\mathbb{Z}$, m: $seq\langle seq\langle\mathbb{Z}\rangle\rangle$) : $\mathbb{Z}$ {

`requiere:` { $|m| > 0$ }

`requiere:` { $|m[0]| > 0$ }

`requiere:` { Todos los elementos de la secuencia m tienen la misma longitud }

`asegura:` {

$resultado = \sum_{i=0}^{|m|-1} \sum_{j=0}^{|m[i]|-1} 1$ si $m[i][j]$ es igual a e, 0 si no }

}

Por ejemplo, `cantidadDeApariciones 1 [[1,0],[0,1]]` es igual a 2.

Posible solución Ej8.2

```
cantidadDeApariciones :: Integer -> [[Integer]] -> Integer
cantidadDeApariciones e [] = 0
cantidadDeApariciones e (x:xs) = cantAparicionesEnFila e x + cantidadDeApariciones e xs

cantAparicionesEnFila :: Integer -> [Integer] -> Integer
cantAparicionesEnFila e [] = 0
cantAparicionesEnFila e (x:xs)
    | x == e = 1 + cantAparicionesEnFila e xs
    | otherwise = cantAparicionesEnFila e xs
```